

# EndField Game

114-2物件導向程式設計實習

B11132017 林科位  
B11115040 葉祈均

# 架構

Board

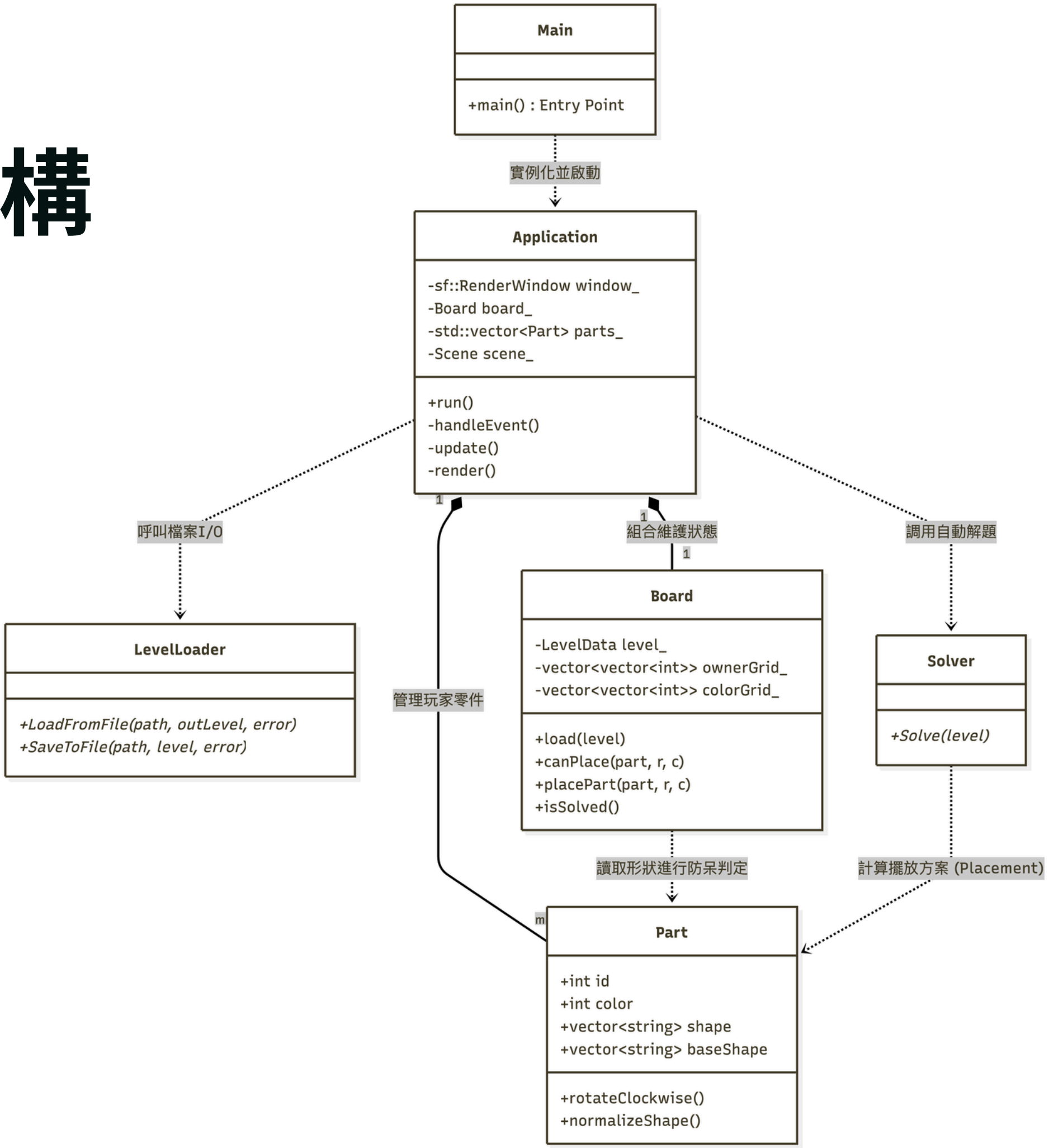
Part

Application

LevelLoader

Solver

# 架構



# 情境：程式主控

```
#include "Application.h"

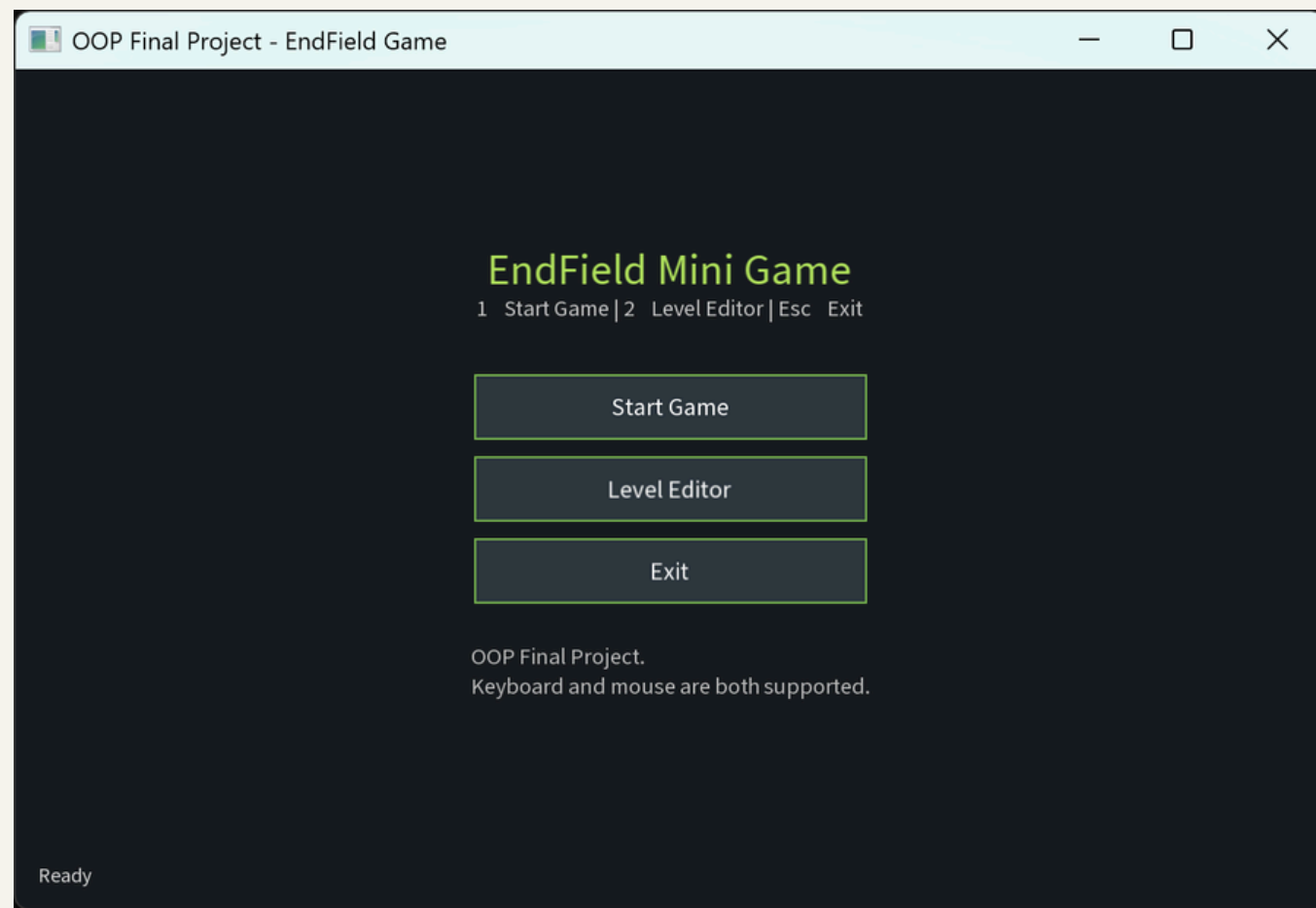
int main() {
    Application app;
    app.run();
    return 0;
}
```

```
void Application::run() {
    while (window_.isOpen()) {
        sf::Event event{};
        while (window_.pollEvent(event)) handleEvent(event);
        update();
        render();
    }
}
```

```
void Application::handleEvent(const sf::Event& event) {
    if (event.type == sf::Event::Closed) { window_.close(); return; }
    if (scene_ == Scene::MainMenu) handleMainEvent(event);
    else if (scene_ == Scene::LevelSelect) handleLevelSelectEvent(event);
    else if (scene_ == Scene::Playing) handlePlayEvent(event);
    else if (scene_ == Scene::Editor) handleEditorEvent(event);
}
```

```
void Application::render() {
    window_.clear(sf::Color(24, 28, 32));
    if (scene_ == Scene::MainMenu) drawMainMenu();
    else if (scene_ == Scene::LevelSelect) drawLevelSelect();
    else if (scene_ == Scene::Playing) drawGame();
    else if (scene_ == Scene::Editor) drawEditor();
    drawText(status_, 24, 780, 18, sf::Color(220,220,220));
    window_.display();
}
```

# 情境：主頁



```
void Application::handleMainEvent(const sf::Event& event) {
    if (event.type != sf::Event::KeyPressed && event.type != sf::Event::MouseButtonPressed) return;
    if (event.type == sf::Event::KeyPressed) {
        if (event.key.code == sf::Keyboard::Num1) scene_ = Scene::LevelSelect;
        if (event.key.code == sf::Keyboard::Num2) { resetEditor(); scene_ = Scene::Editor; }
        if (event.key.code == sf::Keyboard::Escape) window_.close();
    } else {
        auto m = window_.mapPixelToCoords(sf::Mouse::getPosition(window_));
        if (sf::FloatRect(450, 300, 380, 60).contains(m)) scene_ = Scene::LevelSelect;
        if (sf::FloatRect(450, 380, 380, 60).contains(m)) { resetEditor(); scene_ = Scene::Editor; }
        if (sf::FloatRect(450, 460, 380, 60).contains(m)) window_.close();
    }
}
```

# 情境：程式主控

```
#include "Application.h"

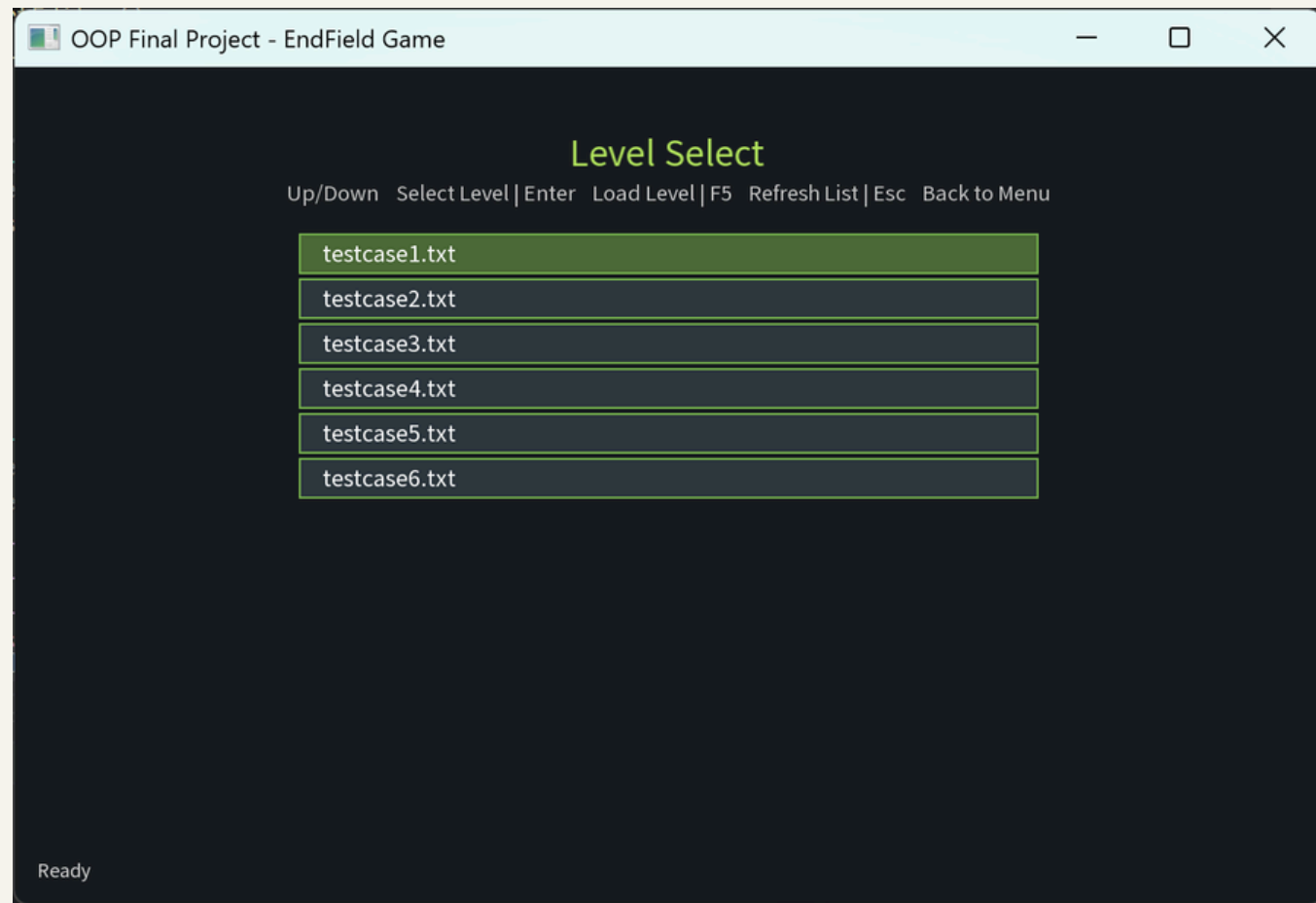
int main() {
    Application app;
    app.run();
    return 0;
}
```

```
void Application::run() {
    while (window_.isOpen()) {
        sf::Event event{};
        while (window_.pollEvent(event)) handleEvent(event);
        update();
        render();
    }
}
```

```
void Application::handleEvent(const sf::Event& event) {
    if (event.type == sf::Event::Closed) { window_.close(); return; }
    if (scene_ == Scene::MainMenu) handleMainEvent(event);
    else if (scene_ == Scene::LevelSelect) handleLevelSelectEvent(event);
    else if (scene_ == Scene::Playing) handlePlayEvent(event);
    else if (scene_ == Scene::Editor) handleEditorEvent(event);
}
```

```
void Application::render() {
    window_.clear(sf::Color(24, 28, 32));
    if (scene_ == Scene::MainMenu) drawMainMenu();
    else if (scene_ == Scene::LevelSelect) drawLevelSelect();
    else if (scene_ == Scene::Playing) drawGame();
    else if (scene_ == Scene::Editor) drawEditor();
    drawText(status_, 24, 780, 18, sf::Color(220, 220, 220));
    window_.display();
}
```

# 情境：讀取關卡

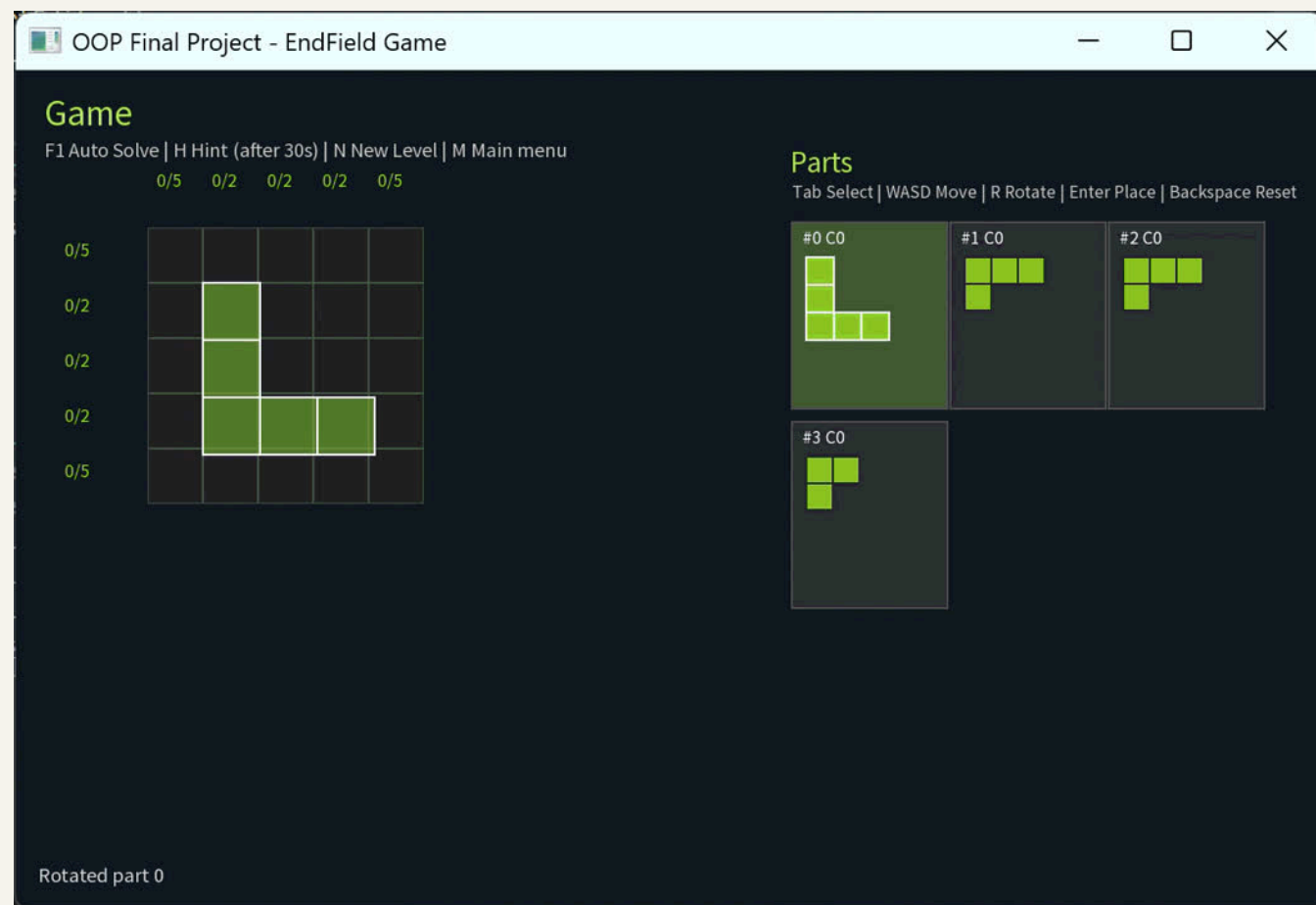


```
void Application::handleLevelSelectEvent(const sf::Event& event) {
    if (event.type == sf::Event::KeyPressed) {
        if (event.key.code == sf::Keyboard::Escape) scene_ = Scene::MainMenu;
        if (event.key.code == sf::Keyboard::Up) selectedLevel_ = std::max(0, selectedLevel_ - 1);
        if (event.key.code == sf::Keyboard::Down) selectedLevel_ = std::min((int)levelFiles_.size() - 1, selectedLevel_ + 1);
        if (event.key.code == sf::Keyboard::Enter) loadSelectedLevel();
        if (event.key.code == sf::Keyboard::F5) { scanLevels(); setStatus("Level list refreshed."); }
    }
    if (event.type == sf::Event::MouseButtonPressed) {
        auto m = window_.mapPixelToCoords(sf::Mouse::getPosition(window_));
        for (int i = 0; i < (int)levelFiles_.size(); ++i) {
            sf::FloatRect row(280, 165 + i * 44.0f, 720, 36);
            if (row.contains(m)) { selectedLevel_ = i; if (event.mouseButton.button == sf::Mouse::Left)
loadSelectedLevel(); }
        }
    }
}
```

```
void Application::loadSelectedLevel() {
    scanLevels();
    if (levelFiles_.empty()) { setStatus("No level file found in assets/levels."); return; }
    selectedLevel_ = std::clamp(selectedLevel_, 0, (int)levelFiles_.size() - 1);
    loadLevelFromPath(levelFiles_[selectedLevel_].string());
}

void Application::loadLevelFromPath(const std::string& path) {
    LevelData level;
    std::string error;
    if (!LevelLoader::LoadFromFile(path, level, error)) { setStatus(error); return; }
    startPlay(level, fs::path(path).filename().string());
}
```

# 情境：遊戲開始



```
void Application::startPlay(const LevelData& level, const std::string& name) {
    currentLevel_ = level;
    parts_ = level.parts;
    board_.load(currentLevel_);
    selectedPart_ = -1;
    dragging_ = false;
    dragPart_ = -1;
    solution_.reset();
    showHint_ = false;
    showSolution_ = false;
    playClock_.restart();
    scene_ = Scene::Playing;
    setStatus("Loaded: " + name);
}
```

# 情境：程式主控

```
#include "Application.h"

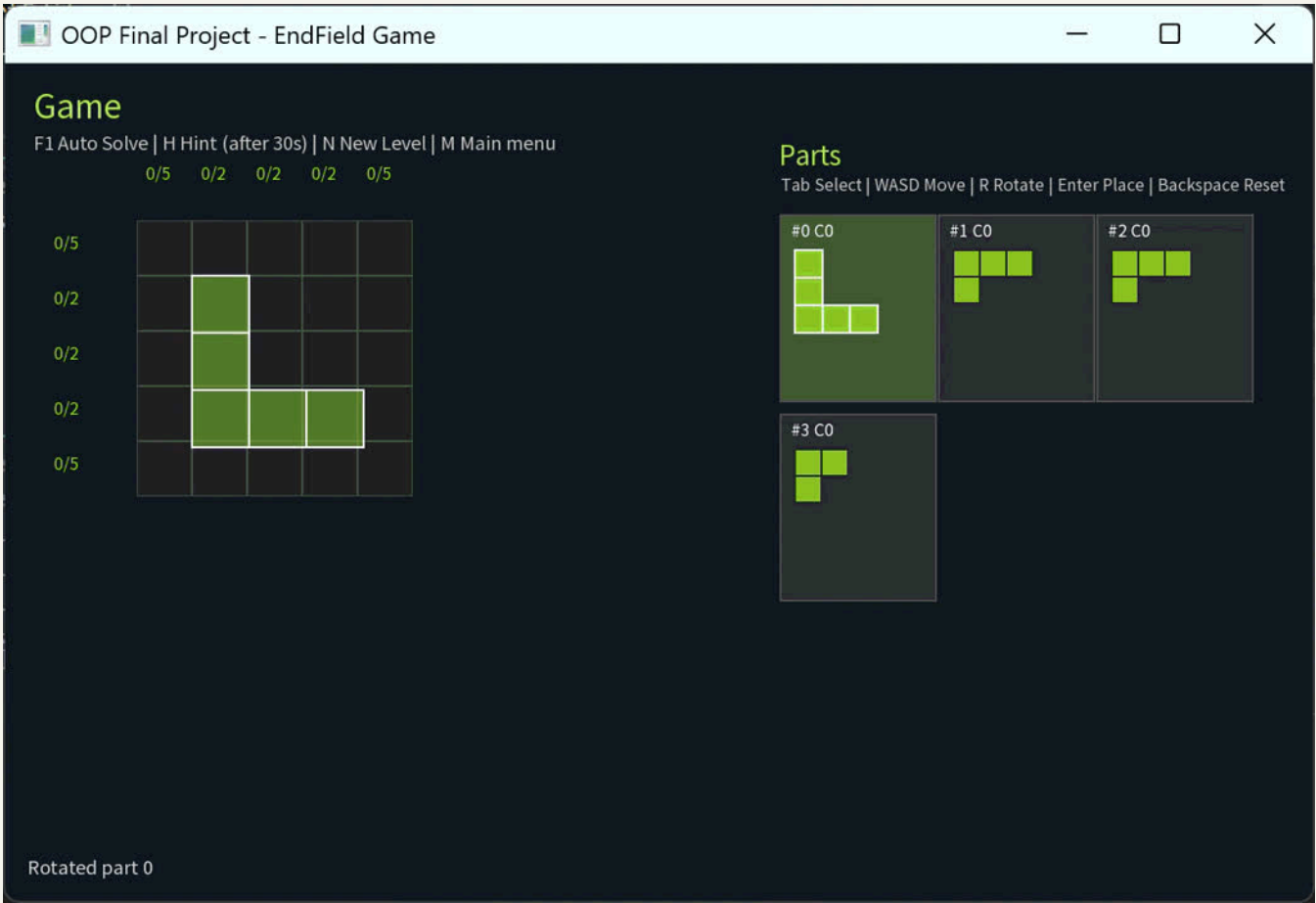
int main() {
    Application app;
    app.run();
    return 0;
}
```

```
void Application::run() {
    while (window_.isOpen()) {
        sf::Event event{};
        while (window_.pollEvent(event)) handleEvent(event);
        update();
        render();
    }
}
```

```
void Application::handleEvent(const sf::Event& event) {
    if (event.type == sf::Event::Closed) { window_.close(); return; }
    if (scene_ == Scene::MainMenu) handleMainEvent(event);
    else if (scene_ == Scene::LevelSelect) handleLevelSelectEvent(event);
    else if (scene_ == Scene::Playing) handlePlayEvent(event);
    else if (scene_ == Scene::Editor) handleEditorEvent(event);
}
```

```
void Application::render() {
    window_.clear(sf::Color(24, 28, 32));
    if (scene_ == Scene::MainMenu) drawMainMenu();
    else if (scene_ == Scene::LevelSelect) drawLevelSelect();
    else if (scene_ == Scene::Playing) drawGame();
    else if (scene_ == Scene::Editor) drawEditor();
    drawText(status_, 24, 780, 18, sf::Color(220, 220, 220));
    window_.display();
}
```

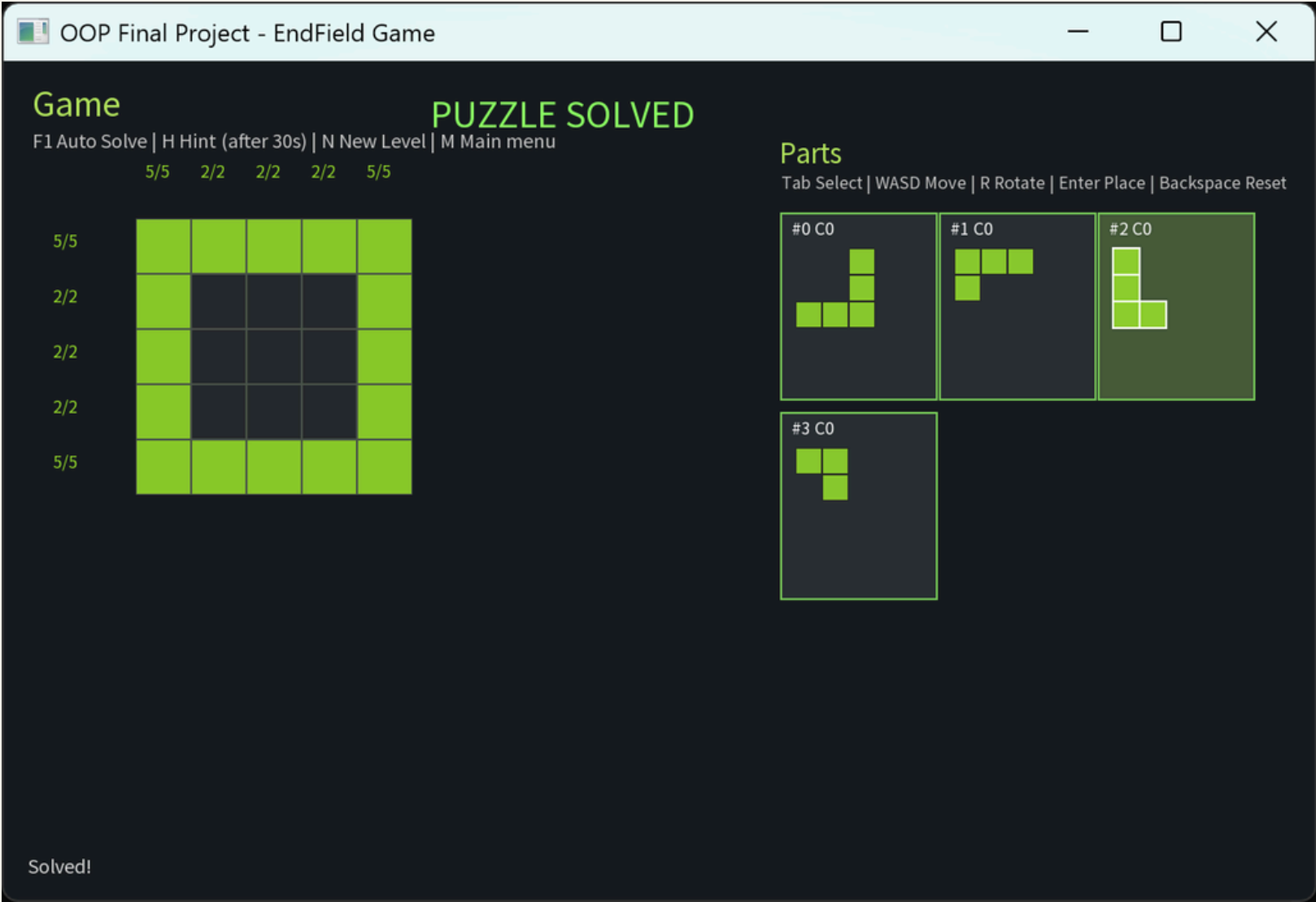
# 情境：遊戲開始



```
void Application::handlePlayEvent(const sf::Event& event)
```

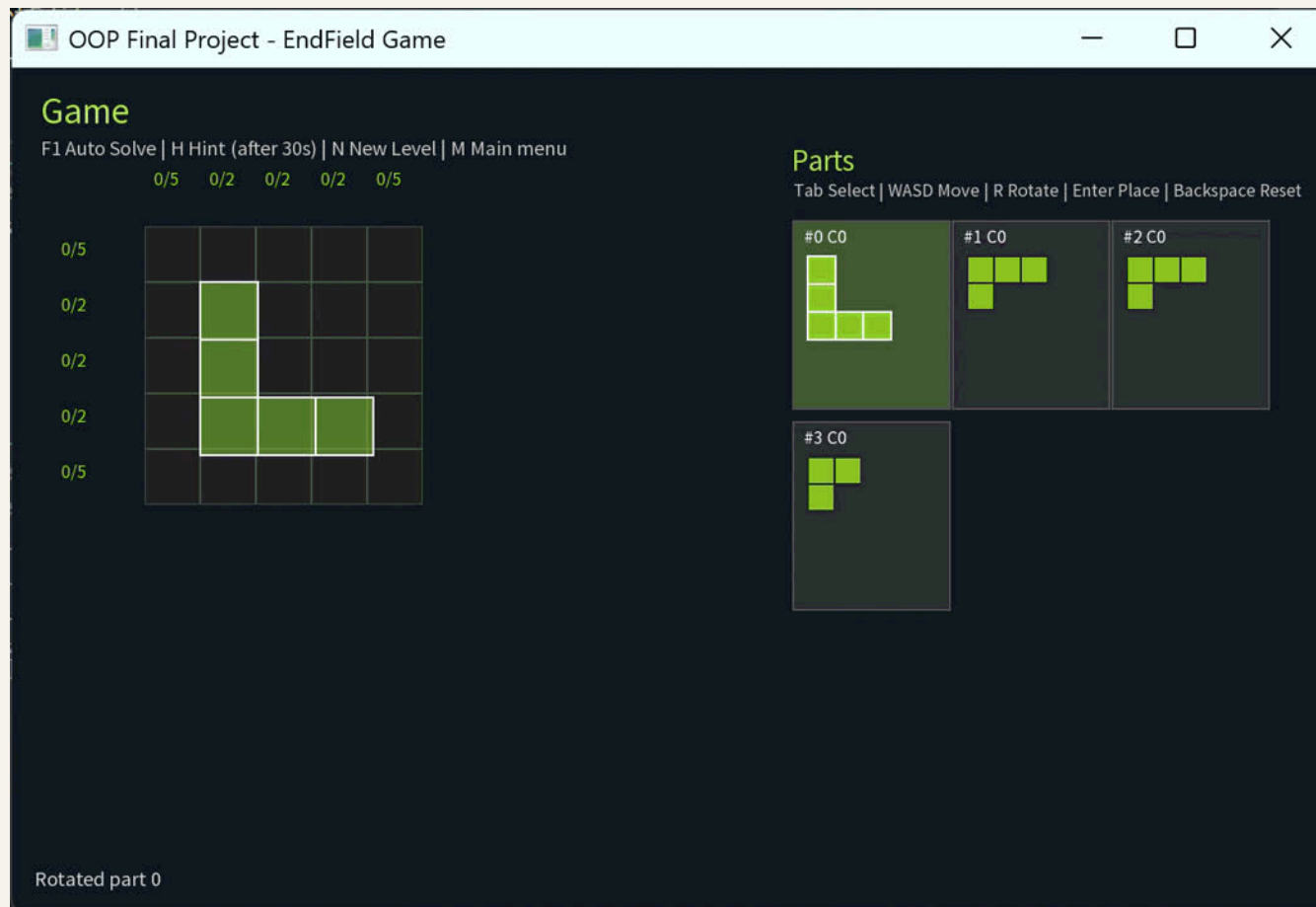
```
void Application::drawGame() {
    drawText("Game", 30, 30, 32, sf::Color(180,230,95));
    drawText("F1 Auto Solve | H Hint (after 30s) | N New Level | M Main menu", 30, 70, 18,
sf::Color(200,200,200));
    drawBoard(130, 155, true);
    drawPartsPanel();
    if (board_.isSolved()) drawText("PUZZLE SOLVED", 420, 40, 34, sf::Color(140,255,100));
}
```

# 情境：解題成功



```
bool Board::isSolved() const {
    auto rowCount = countColorRows();
    auto colCount = countColorCols();
    for (int color = 0; color < level_.colorCount; ++color) {
        for (int r = 0; r < level_.rows; ++r) {
            if (rowCount[color][r] != level_.rowTargets[color][r])
                return false;
        }
        for (int c = 0; c < level_.cols; ++c) {
            if (colCount[color][c] != level_.colTargets[color][c])
                return false;
        }
    }
    return true;
}
```

# 情境：選取和移動



```
auto clampPartInsideBoard = [&](Part& part) {
    int maxRow = std::max(0, board_.rows() - part.rows);
    int maxCol = std::max(0, board_.cols() - part.cols);

    part.row = std::clamp(part.row, 0, maxRow);
    part.col = std::clamp(part.col, 0, maxCol);
};
```

```
if (event.key.code == sf::Keyboard::Tab && !parts_.empty()) {
    selectedPart_ = selectedPart_ < 0 ? 0 : (selectedPart_ + 1) %
static_cast<int>(parts_.size());
}
```

```
if (selectedPart_ >= 0 && selectedPart_ < static_cast<int>(parts_.size())) {
    Part& p = parts_[selectedPart_];
    int nr = p.row;
    int nc = p.col;

    if (event.key.code == sf::Keyboard::W)
        nr--;
    if (event.key.code == sf::Keyboard::S)
        nr++;
    if (event.key.code == sf::Keyboard::A)
        nc--;
    if (event.key.code == sf::Keyboard::D)
        nc++;

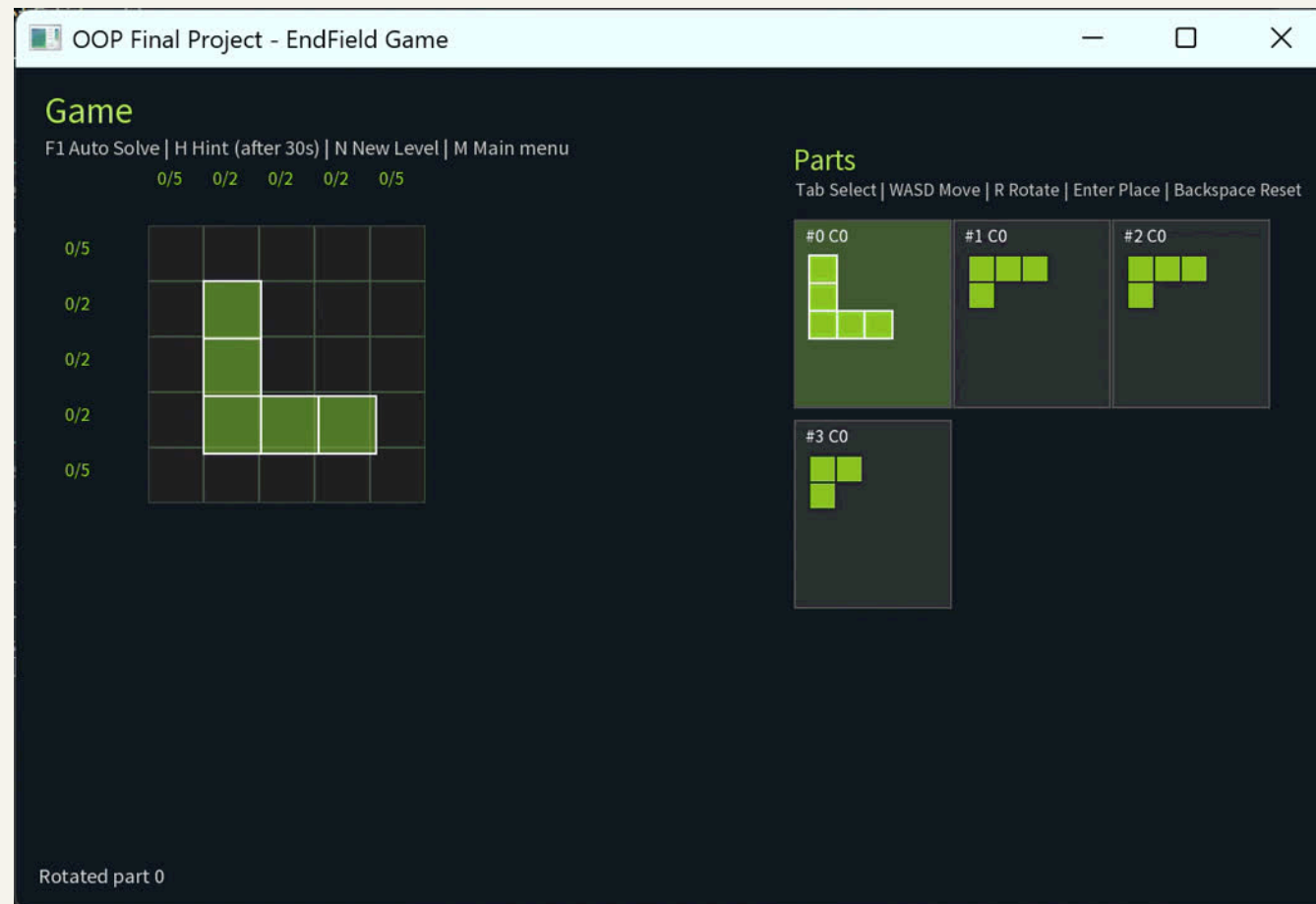
    if (event.key.code == sf::Keyboard::R) {
        board_.removePart(p);
        p.rotateClockwise();
        clampPartInsideBoard(p);
        p.animScale = 1.25f;
        setStatus("Rotated part " + std::to_string(p.id));
    }

    if (event.key.code == sf::Keyboard::W || event.key.code == sf::Keyboard::S ||
event.key.code == sf::Keyboard::A ||
event.key.code == sf::Keyboard::D) {
        board_.removePart(p);
        p.row = nr;
        p.col = nc;
        clampPartInsideBoard(p);
    }

    if (event.key.code == sf::Keyboard::Enter) {
        std::string error;

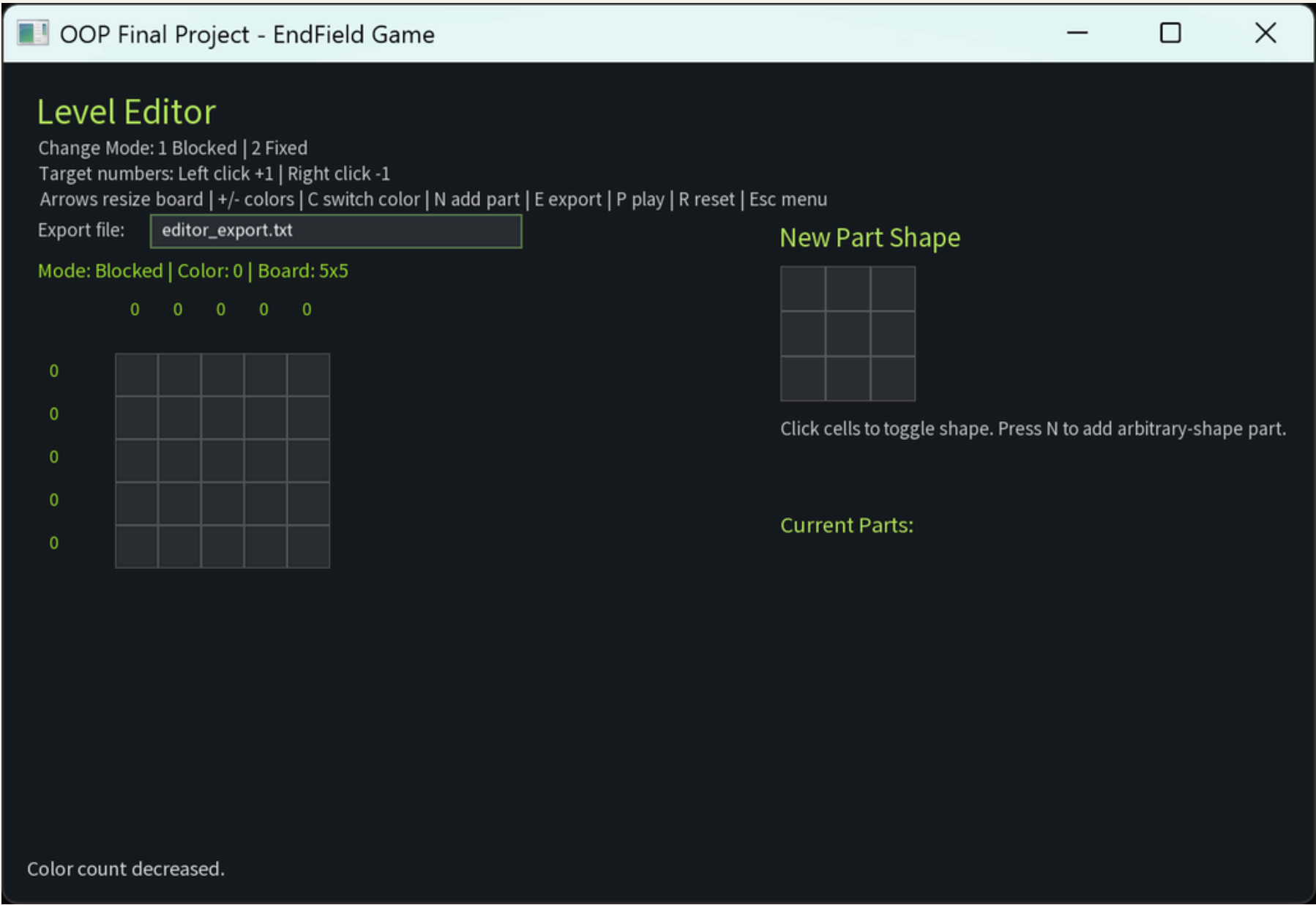
        if (board_.placePart(p, p.row, p.col, error))
            setStatus(board_.isSolved() ? "Solved!" : "Placed part.");
        else
            setStatus(error);
    }
}
```

# 情境：放下part



```
bool Board::placePart(Part& part, int row, int col, std::string& error) {
    removePart(part);
    if (!canPlace(part, row, col, &error)) return false;
    for (auto cell : part.cells()) {
        int rr = row + cell.r;
        int cc = col + cell.c;
        ownerGrid_[rr][cc] = part.id;
        colorGrid_[rr][cc] = part.color;
    }
    part.row = row;
    part.col = col;
    part.placed = true;
    return true;
}
```

# 情境：編輯



```
if (event.key.code == sf::Keyboard::Escape) scene_ = Scene::MainMenu;

if (event.key.code == sf::Keyboard::Num1) editorMode_ = EditorMode::Blocked;
if (event.key.code == sf::Keyboard::Num2) editorMode_ = EditorMode::Fixed;

if (event.key.code == sf::Keyboard::C) {
    editorColor_ = (editorColor_ + 1) % editorLevel_.colorCount;
    setStatus("Editor color changed.");
}
if (event.key.code == sf::Keyboard::Add || event.key.code == sf::Keyboard::Equal) {
    editorLevel_.colorCount = std::min(colorMax, editorLevel_.colorCount + 1);
    setStatus("Color count increased.");
}
if (event.key.code == sf::Keyboard::Hyphen) {
    editorLevel_.colorCount = std::max(colorMin, editorLevel_.colorCount - 1);
    editorColor_ = 0;
    setStatus("Color count decreased.");
}

if (event.key.code == sf::Keyboard::Up) editorLevel_.rows = std::max(boardLengthMin,
editorLevel_.rows - 1);
if (event.key.code == sf::Keyboard::Down) editorLevel_.rows = std::min(boardLengthMax,
editorLevel_.rows + 1);
if (event.key.code == sf::Keyboard::Left) editorLevel_.cols = std::max(boardLengthMin,
editorLevel_.cols - 1);
if (event.key.code == sf::Keyboard::Right) editorLevel_.cols = std::min(boardLengthMax,
editorLevel_.cols + 1);

if (event.key.code == sf::Keyboard::N) {
    bool any = false;

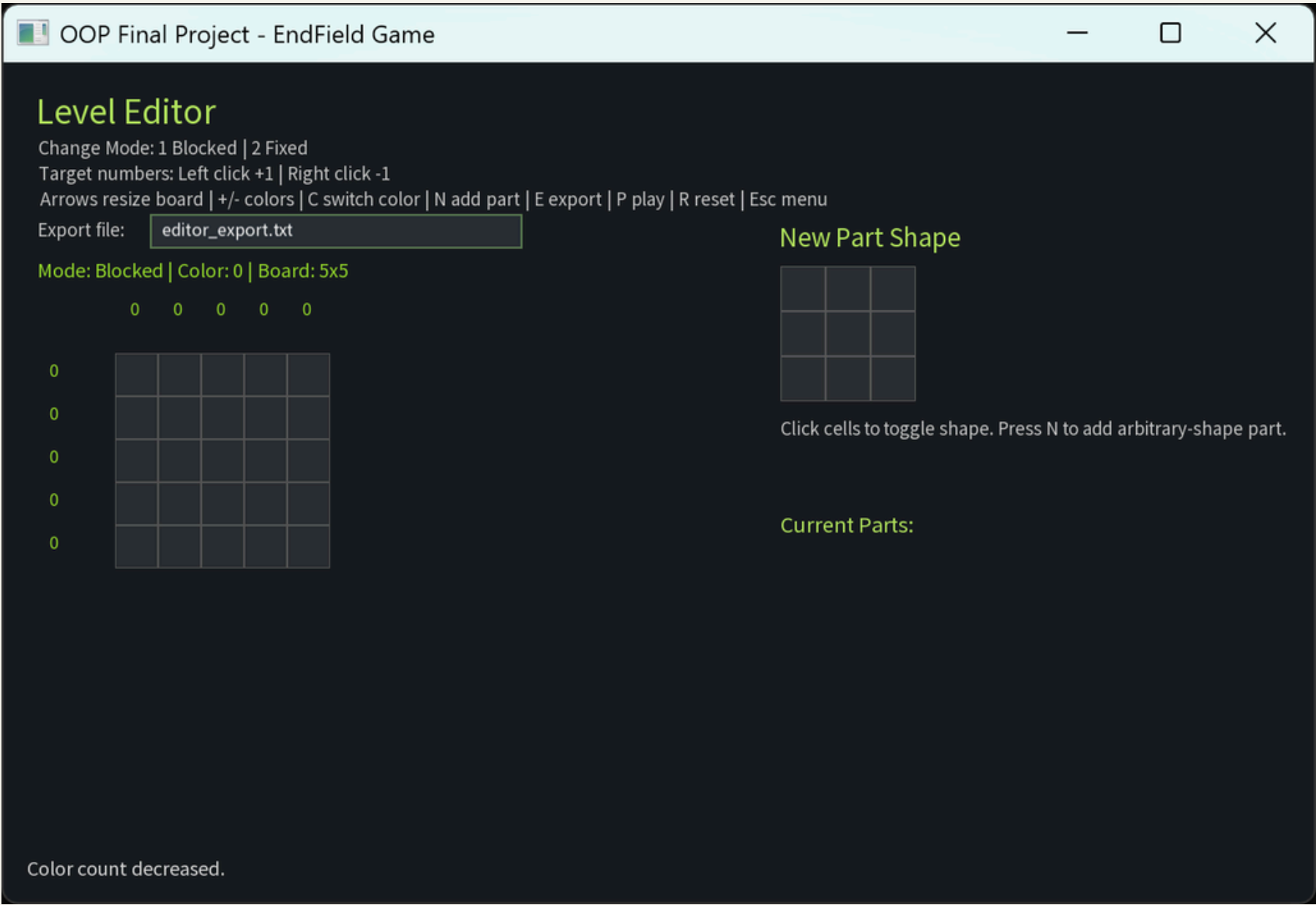
    for (auto& row : editorShape_) {
        for (char ch : row) {
            if (ch == '1')
                any = true;
        }
    }

    if (!any) {
        setStatus("Part shape is empty.");
    } else {
        editorLevel_.parts.emplace_back(static_cast<int>(editorLevel_.parts.size()),
editorColor_, editorShape_);
        editorShape_.assign(editorPartRows_, std::string(editorPartCols_, '0'));
        editorPartListScroll_ = editorPartListTotalHeight();
        clampEditorPartListScroll();

        setStatus("Part added.");
    }
}

if (event.key.code == sf::Keyboard::E) exportEditor();
if (event.key.code == sf::Keyboard::P) playEditorLevel();
if (event.key.code == sf::Keyboard::R) resetEditor();
```

# 情境：編輯



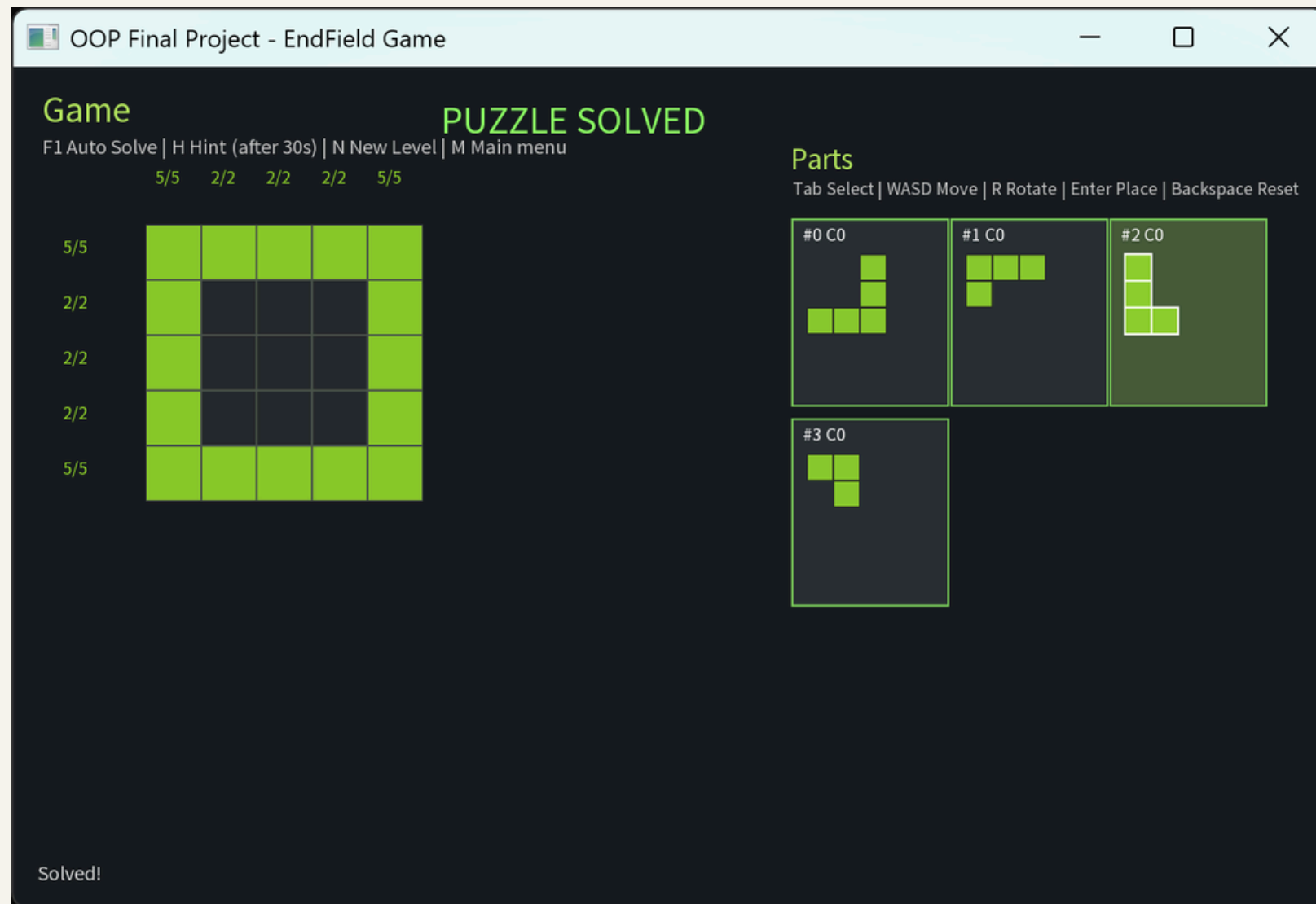
```
auto m = window_.mapPixelToCoords(sf::Mouse::getPosition(window_));
```

```
sf::Vector2f po(760, 200);

int pr = static_cast<int>(std::floor((m.y - po.y) / 44.0f));
int pc = static_cast<int>(std::floor((m.x - po.x) / 44.0f));

if (pr >= 0 && pr < editorPartRows_ && pc >= 0 && pc < editorPartCols_) {
    editorShape_[pr][pc] = editorShape_[pr][pc] == '1' ? '0' : '1';
}
```

# 情境：自動解題



```
std::optional<std::vector<Placement>> Solver::Solve(const LevelData& level)
```

1. 建立暫時計算用的盤面 `occ`
2. 建立 `colorGrid`
3. 建立 `rowCount / colCount`
4. 先放入 `blocked / fixed cell`
5. 檢查 `fixed cell` 是否已經超過目標
6. 將零件依照 `cellCount` 由大到小排序
7. 對每個零件嘗試所有不重複旋轉
8. 對每個旋轉形狀嘗試所有盤面位置
9. 如果合法就放下去，繼續 DFS
10. 若後續失敗就回溯
11. 所有零件都放完後，檢查 `row / col` 是否完全符合

# How to implement with AI

1

丟要求  
請他給出此專案基本架構



114-2OOP實習\_期末project評分標準....  
PDF

這是我的作業要求  
我想要先依照之前的Project1 建立一個CMake專案 並一樣用SFML做GUI  
先幫我建立CMakeLists.txt和其他基本所需的檔案 我可以正常建置後再開始實際做project

2

根據每個小細節下prompt調整GUI或顯示邏輯



src.zip  
壓縮封存檔案

剛剛那個改好了 現在我有以下需求

1. 當我選好level並進入時 初始狀態應該要是未選取任何part 而不是自動選取第一個part(如第一張圖所示)
2. 我用滑鼠拖拉part的時候會像第二張圖那樣 可以放在Board的外面 不可以這樣
3. 我用鍵盤移動part的時候會像第三張圖那樣 只要左上角沒有超過邊界就可以繼續往右/往下 我要的是只要有part的任何cell超過邊界就不能再移動過去



src.zip  
壓縮封存檔案

我有幾個地方想修正

1. 當我在移動Part的時候 按下ESC要將他從盤面上移除 「並恢復到未選取part的狀態」 現在是按了Esc還是停留在選取狀態(如第一張圖)
2. 當我在加入Fixed的時候 RowTarget和ColTarget的數字欄應該同步更新 並且為其更新最小值  
例如像我的第二張圖 第一列和第二行都應該同步更新成1 並且最小值更新成1而非0

OOP FINAL

# The End

THANK YOU FOR LISTENING